

Recording - 2025/04/14 18:49 CEST - Transcript

Attendees

João Torres

Transcript

João Torres:

Slide 1: Hello, I'll be presenting to you the work performed during the machine learning module in my individual project, where I implemented CNN for image recognition.

Slide 2: Here's a glimpse of the agenda for this presentation. I'll be starting by presenting the data set used and the goal of the implementation overall. Then, I'll proceed to talk a little bit about the architecture and activation and loss functions. Then, I'll talk about the data pipeline. Speaking about data proceeding then to the training process and validation strategy and finally showing the tests that have been done to the model and completing the presentation with both critical analysis and also insights and reflections on what has been achieved during

Slide 3: The goal of this project was to implement an NN-based Image Classifier. To do so, CIFAR-10, a data set designed for Image Classification, has been used. It comprises 60,000 colour images with a size of 32x32 pixels. 10,000 of those images are designated for testing. The data set is well-balanced, with 6,000 images from each class.

Slide 4: The architecture used was created 2 years ago by Farzad Nekouei, and it is inspired by the well-known VGG16 architecture. The network begins with a pair of Conv2D layers, which are convoluted layers each with 32 filters of size 3x3. In a convoluted layer, specialised filters known as kernels are applied and traverse through the input image to learn complex visual patterns. This is followed by a Batch Normalization layer which accelerates training and provides some level of regularization, helping to prevent overfitting. Afterwards, there's a MaxPooling2D layer, which reduces the spatial dimensions (height and width), effectively providing a form of translation invariance and reducing computational complexity. This is followed by a Dropout layer that randomly sets a fraction (0.2 for the first dropout layer) of the input units to 0 at each update during training, helping to prevent overfitting. This set-up of convoluted layer pairs with batch normalisation, followed by max pooling and dropout, is repeated three more times until a total of 16 convoluted layers exist. The difference between the pairs is that the number of filters doubles from 32 to 64 to 128, concluding at 256 filters. The increasing number of filters allows the network to learn increasingly complex features. The network concludes with 2 final Layers: 1) First, a Flatten Layer is used to convert the output from the last layers from 2D to a one-dimensional vector, 2) Finally, the Fully Connected Layer is used for classification, comprising 10 units: one for each class of CIFAR-10. In total, the model has 1,186,346 parameters, of which 1,184,426 are trainable (Nekouei, 2023)

Slide 5: There are 2 activation functions used: 1) ReLU for convoluted layers. ReLU (short for Rectified Linear Units) is very commonly used in CNNs, and it works by outputting the input directly if it's positive and zero otherwise, it introduces non-linearity efficiently. It has computational speed as an advantage. 2) The softmax function is used in the final architecture layer, the Fully Connected Layer, because it converts raw scores (logits) into a probability distribution where all outputs sum to 1.0 and each value is between

0 and 1. This makes it ideal for multi-class classification tasks, such as our use case 3) As a loss function, categorical cross entropy has been used. This function measures the difference between the predicted probability distribution (from the softmax layer) and the true, one-hot encoded distribution of the labels. It excels in CIFAR 10 because it penalizes the model when it is confident about the wrong class, encouraging it to produce accurate probability distributions that match the true labels.

Slide 6: The data has been partitioned into three different validation and testing. The training set in this case seven almost 71% of the total images are used specifically for training the model. When it comes to validation set it has been created using the split function from the scikitle learn package. It is roughly 12.5% of the total, and it's used specifically to evaluate the model during training and helps with detecting and preventing overfitting. Finally, the testing set is used when training is complete and in this case it comprises 7% of the total images.

Slide 7: A data augmentation pipeline has been tested having been built three scenarios. No augmentation, intermediate augmentation which comprised five different attributes including rotate rotation of up to 15° horizontal flip, zoom, brightness manipulation, shear and channel shifting. And all of these have been used because they yielded considerable accuracy gains in other examples in the literature also using CIFAR 10 data set (Shorten & Khoshgoftaar, 2019). The last scenario was what we called full augmentation and this really builds on the intermediate augmentation and includes two additional operations width and weight shifting. The goal of this is to introduce additional variability in the data set and evaluate whether it helps the model prevent overfitting (Nekouei, 2023).

Slide 8: Several runs of the model have been performed, testing for variability in hyperparameters. Batch sizes of 64, 128, and 256 have been tested. Whether shuffling is added or not has also been tested, and finally epoch number and speed have also been evaluated in different configurations.

Slide 9: The validation strategy for model training was as follows. First, 15% of the training set has been used for validation, as mentioned before. An early stopping function has been added, which monitors validation loss and interrupts the model training if no improvements are found in 40 epochs, restoring the weights to the best model run until that moment. Finally, a reduce learning rate on plateau function has also been used, which adds important dynamicity to the learning rate hyperparameter. It reduces this hyperparameter value by half in case the loss value plateaus for 10 epochs until the preset minimum value for learning rate is reached.

Slide 10: Accuracy has been picked as the main model evaluation metric because the data set is balanced, meaning it has a similar number of samples per class, and all classes are equally important.

00:10:00

João Torres: Indeed, for CIFAR 10, accuracy works well because it has exactly 6,000 images per class in the training set and 1,000 images per class in the test set, making it perfectly balanced. The different tests performed gather excellent insights on the model. The first test has been performed by omitting the data augmentation step and yielded the highest value of training accuracy, but validation and testing accuracies differ by 10% when compared to the accuracy. This stands as a strong indicator that the model is overfitting to the training data. Hence, for subsequent tests, the decision has been made to always include data augmentation. In the next three tests, different batch sizes have been used, starting with 256 and reducing it by half in each subsequent test, ending at 64 for test number four. The difference in average epoch length is significant and significantly shorter for a very large batch size, which is

expected as the model takes less time to run through the entire data set because it does it in larger chunks. Even so, the final accuracy is smaller.

Having the 256 batch size example yielded the lowest overall training accuracy in all the tests performed. Either way, the performance gain could be worth it if the use case doesn't require a very high accuracy. The best result in terms of accuracy has been achieved for a batch size of 64. Hence, this parameter has been set for this value and has been used for the.

In test number five, additional data augmentation is used, having yielded the best performance overall, achieving a 90.11% accuracy in the testing data set, which is very high.

In test number six, the impact of shuffling was tested. Shuffling is the process of randomly reordering data between epochs of model training. It's useful to make sure that the model is not learning insights based on the order of the data. Even though with shuffling in this test, the model performed worse in accuracy than the previous test, it needed overall 100 fewer epochs to run, adding just 1 second per epoch, which is a very significant number of epochs and hence time saved, which could be considered.

Slide 11:

João Torres: The graphs in this slide have been built using the evaluate function from the keras package and depict the evolution of loss value and accuracy for the 300 epochs needed in the highest performance tests. As mentioned before, a patch size of 64 has been used, and the highest data augmentation setup has been used. The result is an accuracy of about 90% across all three data sets and also with a small difference in accuracy between them, which hints towards a low overfitting setting.

Slide 12:

Three additional tests have been performed with three images outside of the safe 10 data set. Two dogs and one frog. The model correctly predicted the frog but wrongly classified the dogs as being frogs as well. Additional tests would need to be performed to confidently get to a reason for the misclassification. But, the preprocessing to fit the 32x32 size requirement using the CV2 function could be one of the reasons as key image attributes could be lost in such an extreme image resizing processing.

Slide 13: as per challenges and limitations for this project.

00:15:00

The first challenge is that the epoch runtime has been big, with model runs sometimes taking two to three hours to complete, and this limits the time and how many variations can be tested in this project. Another limitation is that between different tasks, it is challenging to individualize defects of small hyperparameter tweaks, making it hard to get to a really optimal setup. The last limitation that we found was that the model didn't qualify or classify images outside of the scar 10 data set. As for future improvements, the following could be considered: First, new and more complex data augmentation techniques such as patch shuffle regularization, symbol pairing and random erasing could be tested as they have yielded a very good accuracy gain in the literature (Shorten & Khoshgoftaar, 2019). Second, tensor slicing could be added to the training and validation data sets, as this has been set as a technique with a very significant performance gain. Better GPUs could be rented to perform more test variations as opposed to using the Apple M2 GPU as it has been done in this project. Another model implementation in different data sets could be envisioned, such as the CIFAR 100 or the AlexNet image data sets. If the GPU

computation power is rented and more computing power is available, new architectures could be attempted, such as the deeper, more complex architectures like ResNet 50.

Slide 14: As a reflection coming out of this project, it has been invaluable to gain skills in setting up a neural network architecture and understanding the different components and how they contribute to different model outputs and performance. Indeed, it became clear while building the image augmentation pipeline and designing different tests why data pipelining and augmentation are critical when training image-classifying models, as they had a very significant impact. Second, performance and speed as important components of an implementation as opposed to only taking accuracy or really any other performance metric into account. If the speed of running the model is very significantly smaller, we observed in our test with a batch size of 256 where the time per epoch was two to three times smaller, it could be well worth the accuracy compromise in those cases.

Slide 15: There are numerous applications for image classification algorithms in the real world, for example, being used successfully in medical imaging, detecting early-stage tumors in MRI images and self-driving settings for detecting obstacles, among many other settings. Perhaps a more straightforward use case I would like to explore would be to create an improved iOS where newly taken photos are automatically classified and divided into folders, really enhancing the user experience in this application. Thank you so much

Meeting ended after 00:19:46 🙌

This editable transcript was computer generated and might contain errors. People can also change the text after it was created.

Reference List:

Nekouei, F. (2023) CIFAR-10 Image Classification with CNN. Available at: <https://www.kaggle.com/code/farzadnekouei/cifar-10-image-classification-with-cnn> [Accessed: 14 April 2025]

Shorten, C. & Khoshgoftaar, T.M. (2019) A survey on Image Data Augmentation for Deep Learning. J Big Data 6, 60 . <https://doi.org/10.1186/s40537-019-0197-0>